

SMART CAMPUS EVENT MANAGEMENT SYSTEM

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering
By**

Neelam Sai Karthikeya (VTU26644)

*10211CS224
Full Stack Application Development
Slot : E1*



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled "SMART CAMPUS EVENT MANAGEMENT SYSTEM" by Neelam Sai Karthikeya (VTU26644) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr.Hemamalini.S

AssistantProfessor(SeniorGrade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Coordinator

Dr.M.Sumithra

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Neelam Sai Karthikeya

Date:06/05/2026

ABSTRACT

The Smart Campus Event Management Application is a full stack web-based system designed to streamline the organization, registration, and management of campus events. Traditional event handling methods often rely on manual processes, leading to inefficiencies, data loss, and poor user experience. The application enables users to browse events, select ticket types, and complete registrations through an interactive and user-friendly interface. It incorporates dynamic frontend technologies for seamless navigation and backend services for secure data handling and storage. The system ensures accurate record keeping by storing participant details, ticket information, and booking confirmations in a structured database. It also enhances user engagement by offering instant confirmations and a structured booking experience. Overall, the project demonstrates how modern web technologies can be leveraged to build scalable, efficient, and user-centric solutions for campus event management.

Keywords: Event Management, Spring Boot, MVC, REST API, JPA, Thymeleaf, MySQL, Spring Security, CRUD, Web Application.

LIST OF FIGURES

1.1	System Overview Diagram	4
4.1	Project Architecture	14
4.2	Data Flow Diagram	16
5.1	Student Login Page	21
5.2	Event Registration Page	22
5.3	Upcoming Events	23
5.4	E-mail OTP Verification	24
5.5	Add new Event	25

LIST OF TABLES

5.1	Application Feature Comparison	20
7.1	Mapping of Project to Complex Engineering Problems (CEP)	28
7.2	Mapping of Project to Complex Engineering Activities (CEA)	29
7.3	SDG Mapping for the Project	30

LIST OF ACRONYMS AND ABBREVIATIONS

DFD	Data Flow Diagram
MVC	Model View Controller
H2	H2 Database Engine
API	Application Programming Interface
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JPA	Java Persistence API
ORM	Object Relational Mapping
SQL	Structured Query Language
CRUD	Create, Read, Update, Delete
JSP	Java Server Pages
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
IDE	Integrated Development Environment
OOP	Object Oriented Programming

TABLE OF CONTENTS

LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	2
1.3 Scope of the Project	2
1.4 Framework	3
2 SURVEY OF SIMILAR APPLICATION IN MARKET	5
3 FRAMEWORK DESCRIPTION	7
3.1 Frontend Implementation	7
3.2 Backend Implementation	9
3.3 Database Implementation	11

4	METHODOLOGIES	13
4.1	Project Architecture	13
4.2	Data Flow Diagram	15
4.3	Module 1: User Module	17
4.4	Module 2:Admin Module	17
4.5	Module 3:Database Module	17
5	RESULTS AND DISCUSSIONS	19
5.1	Student Login	21
5.2	Registration Page	22
5.3	Upcoming Events	23
5.4	OTP Verification	24
5.5	Add new Event	25
6	CONCLUSION AND FUTURE ENHANCEMENTS	26
6.1	Conclusion	26
6.2	Future Enhancements	27
7	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	28
7.1	Mapping of Project to Complex Engineering Problems (CEP)	28

7.2	Mapping of the Project to Complex Engineering Activities (CEA)	29
7.3	SDG Mapping	30

Chapter 1

INTRODUCTION

1.1 Introduction

The Smart Campus Event Management System is a web-based application developed to simplify and automate the management of campus events such as workshops, seminars, and technical programs. In traditional systems, event management is often handled manually through notice boards or spreadsheets, leading to inefficiencies, lack of coordination, and difficulty in tracking participation. This system provides a centralized digital platform where students can easily browse upcoming events, apply filters, register online, and track their participation history, while administrators can efficiently create, update, and delete event details, as well as monitor registrations and generate reports. The system incorporates modern technologies such as Spring Boot, Spring MVC, and MySQL to ensure efficient backend processing, while Thymeleaf and web technologies provide a dynamic and

user-friendly interface. It also includes features like input validation, secure authentication, and role-based access control to enhance reliability and data security. By enabling real-time updates and streamlined communication between users and administrators, the system improves overall campus engagement and operational efficiency. Thus, it serves as a scalable and effective solution for managing academic events in a structured and organized manner.

1.2 Aim of the Project

The aim of this project is to design and develop an efficient and user-friendly platform that automates the process of managing campus events. The system aims to reduce manual effort, improve event visibility, and enable seamless interaction between students and administrators. It also focuses on providing secure access, accurate data management, and real-time updates for better decision-making.

1.3 Scope of the Project

The scope of the Smart Campus Event Management System includes event creation, management, and participation tracking within a campus environment. Students can view upcoming events, apply filters based on criteria such as date, department, and event type, and regis-

ter for events through an online interface. Administrators are provided with functionalities to perform CRUD (Create, Read, Update, Delete) operations on events, manage event details, and analyze registration data for better decision-making. The system ensures efficient handling of event-related information and improves coordination between students and organizers. Currently, the system is limited to web-based access; however, it can be further extended to support mobile applications, integration of online payment gateways for paid events, real-time notifications, and advanced analytics for enhanced insights and scalability.

1.4 Framework

The system is developed using the Spring Boot framework, which provides a robust backend environment for building scalable web applications. Spring MVC is used to implement the Model-View-Controller architecture, ensuring a clean separation of concerns. Spring Data JPA is utilized for efficient database interaction, while Thymeleaf is used for frontend development to create dynamic web pages. MySQL serves as the database for storing application data, and Spring Security is implemented to ensure authentication and authorization. This framework combination ensures the system is secure, maintainable,

and scalable.

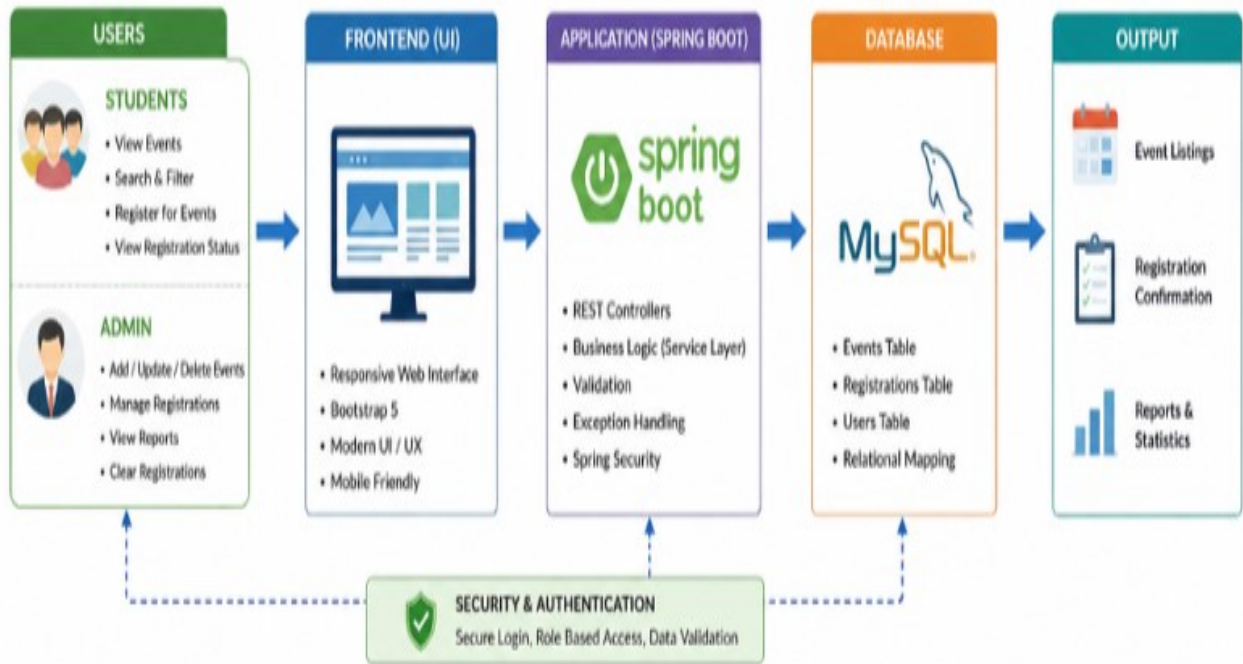


Figure 1.1: System Overview Diagram

[Figure 1.1] shows the overall architecture of the system, illustrating how different components interact to manage campus events efficiently. Users are divided into students and administrators, where students can view, search, and register for events, while administrators handle event creation, updates, and registrations. The frontend provides a responsive user interface built with modern UI technologies, ensuring accessibility across devices. The backend, developed using Spring Boot, manages business logic through REST controllers, validation, and exception handling.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

The increasing need for efficient event management systems has led to the development of several applications that simplify event organization, registration, and communication[1]. Various platforms such as Eventbrite, Meetup, and campus-specific management tools are widely used to manage events in different domains[3]. These applications provide features like event creation, online registration, notifications, and participant tracking[1]. However, most of these platforms are designed for large-scale public events and may not fully address the specific requirements of academic institutions[2].

In the context of educational campuses, event management is often handled using manual processes or basic digital tools such as spreadsheets and notice boards, which can lead to inefficiencies, lack of real-time updates, and poor coordination[2]. Some institutions use cus-

tomized systems, but they may lack advanced features such as filtering options, user-friendly interfaces, and proper analytics[3].

Existing systems also face limitations in terms of security, scalability, and integration with modern web technologies[4]. Many applications do not provide role-based access control or fail to offer a seamless experience for both students and administrators[5]. Additionally, they may lack features like event categorization, department-based filtering, and detailed registration statistics[5].

The Smart Campus Event Management System is designed to overcome these limitations by providing a dedicated and comprehensive platform tailored specifically for academic environments[6]. It introduces features such as secure user authentication, role-based access control, efficient event management, seamless event registration, and real-time data analysis[4]. The system also supports filtering of events based on multiple criteria, ensuring a personalized and user-friendly experience[7]. By leveraging modern technologies such as Spring Boot, Spring MVC, Spring Data JPA, and Thymeleaf, the proposed system ensures high performance, scalability, maintainability, and enhanced user interaction[8]. Overall, the system aims to improve campus engagement, streamline event operations, and provide a reliable and efficient solution compared to existing applications[7].

Chapter 3

FRAMEWORK DESCRIPTION

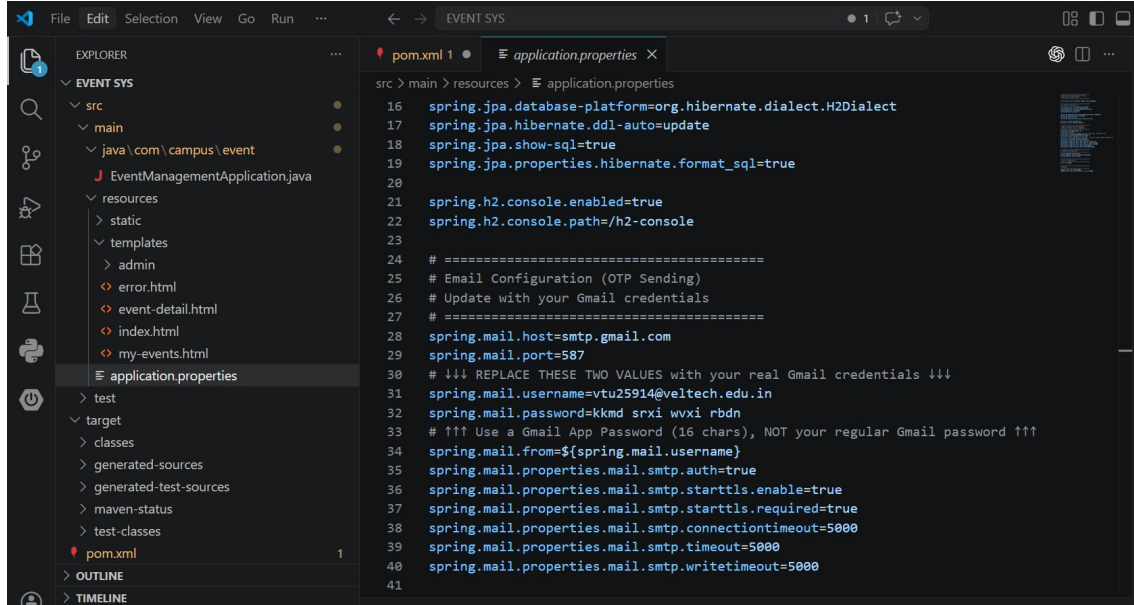
3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend of the Smart Campus Event Management System is developed using HTML5, CSS3, and Thymeleaf[9]. The development environment includes tools such as Visual Studio Code for coding and browser-based testing[9]. Thymeleaf is used as a server-side template engine that integrates seamlessly with Spring Boot, enabling dynamic content rendering[8]. Basic styling is implemented using CSS to ensure a clean and responsive user interface. The use of reusable templates and modular design in Thymeleaf helps maintain consistency across different pages and reduces code duplication[6]. Proper integration between frontend and backend ensures that data is displayed in real-time, allowing users to view updated event information instantly[4]. Overall, the frontend is designed to be intuitive, efficient,

and visually appealing, providing a seamless interaction experience for both students and administrators[8].

3.1.2 Structure of the Project



The above image shows the Visual Studio Code environment with the project structure of the event management system. The application.properties file is open, displaying configurations for database settings, H2 console, and email (SMTP) setup for sending notifications. This file is used to manage application-level properties and ensure proper backend functionality.

3.1.3 Execution Procedure

The frontend is executed through the Spring Boot application. When the server runs, Thymeleaf templates are rendered dynamically based on user requests handled by controllers. Users can access the application via a web browser using the local server URL (e.g., <http://localhost:8080>)

). Navigation between pages is handled through controller mappings, and user inputs are processed via forms integrated with backend services.

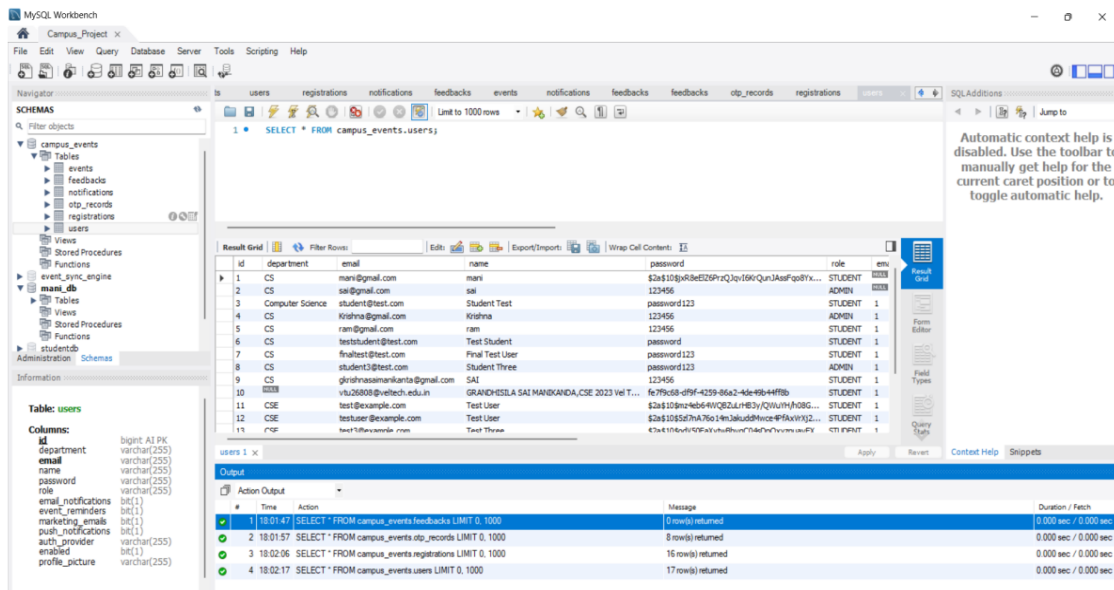
3.2 Backend Implementation

3.2.1 Environment Setup

The backend is developed using Java and the Spring Boot framework. The development environment includes JDK 17 or higher, Maven for dependency management, and an IDE such as IntelliJ IDEA or VS Code[7]. MySQL is used as the database, and it is configured through the application.properties file. Dependencies such as Spring Web, Spring Data JPA, Spring Security, and Validation are included to support backend functionalities[3]. The backend follows the Model-View-Controller (MVC) architecture, where controllers handle incoming HTTP requests, services manage business logic, and repositories interact with the database.[8] Spring Data JPA simplifies database operations by providing built-in methods for CRUD operations, while Hibernate acts as the ORM tool to map Java objects to database tables[6]. Spring Security is implemented to ensure authentication and role-based authorization, protecting sensitive data and restricting access to authorized users only[9]. The system is designed to handle mul-

multiple requests efficiently, ensuring high performance, scalability, and maintainability. Overall, the backend plays a crucial role in processing user requests, managing data, and ensuring secure communication between different components of the application.

3.2.2 Structure of the Project



The above image shows the MySQL Workbench interface displaying the campus events database, where tables such as users, events, registrations, and feedbacks are organized. The query editor is used to execute SQL commands, and the result grid presents stored data like user details, roles, and credentials. This interface helps in managing, viewing, and updating the database records efficiently.

3.2.3 Execution Procedure

The backend application is executed using Maven commands such as `mvn spring-boot:run`. Once the server starts, it listens for HTTP

requests on a specified port (default: 8080). The controllers process incoming requests, interact with the service and repository layers, and return responses to the frontend. Database operations are performed using JPA, and data is stored and retrieved from MySQL. Security mechanisms ensure that only authenticated users can access protected resources.

3.3 Database Implementation

3.3.1 Database Configuration

The Smart Campus Event Management System uses MySQL as the relational database for storing and managing application data[4]. The database is configured in the `application.properties` file of the Spring Boot project, where parameters such as database URL, username, password, and driver class are specified. Spring Data JPA is used to interact with the database, enabling automatic table creation and simplified CRUD operations[7]. Hibernate is used as the ORM (Object Relational Mapping) tool, which maps Java entity classes to database tables. The configuration ensures efficient data handling, scalability, and easy integration with the backend.

3.1.2 Schema Structure

The database schema is designed to support the core functionalities

of the system, including user management, event management, and event registration. The main entities in the schema are User, Event, and Registration. The User entity stores information about students and administrators, including login credentials and roles. The Event entity contains details such as event title, description, date, department, and type. The Registration entity maintains the relationship between users and events, recording which user has registered for which event. The schema follows relational database principles with proper primary and foreign key constraints to maintain data integrity.

3.1.3 Tables created

The database includes the following main tables:

- **User Table:** Stores user details such as user ID, name, email, password, and role (student/admin).
- **Event Table:** Contains event-related information including event ID, title, description, date, department, and type.
- **Registration Table:** Maintains records of event registrations, including registration ID, user ID, and event ID.

Relationships are established using foreign keys, where the Registration table references both User and Event tables. This structure ensures efficient data retrieval and supports features like event participation tracking and statistical analysis.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The Smart Campus Event Management System follows a layered architecture where the Presentation Layer (frontend)—built using HTML, CSS, Bootstrap, and JSP—interacts with the backend through HTTP requests. The core Application Layer is divided into three parts: the Controller Layer, which handles requests and responses; the Service Layer, which contains business logic, validations, and transaction management; and the Repository Layer, which performs data access and CRUD operations using Spring Data JPA. This application connects to the Data Layer, consisting of an H2 in-memory database (eventdb) that stores information such as users and events. Supporting all layers are cross-cutting concerns like Spring Security for authentication and authorization, logging for monitoring, and global exception handling, while the overall system is built using technologies including Spring

Boot, Spring MVC, Spring Data JPA, H2, HTML5, CSS3, Bootstrap, and JSP. Additionally, the architecture includes cross-cutting concerns like Spring Security for authentication and authorization, logging mechanisms to track system activities, and global exception handling to manage errors consistently across the application. The overall design ensures modularity, scalability, maintainability, and ease of testing, while the chosen technology stack—including Spring Boot, Spring MVC, Spring Data JPA, H2, HTML5, CSS3, Bootstrap, and JSP—supports rapid development and efficient deployment of modern web applications.

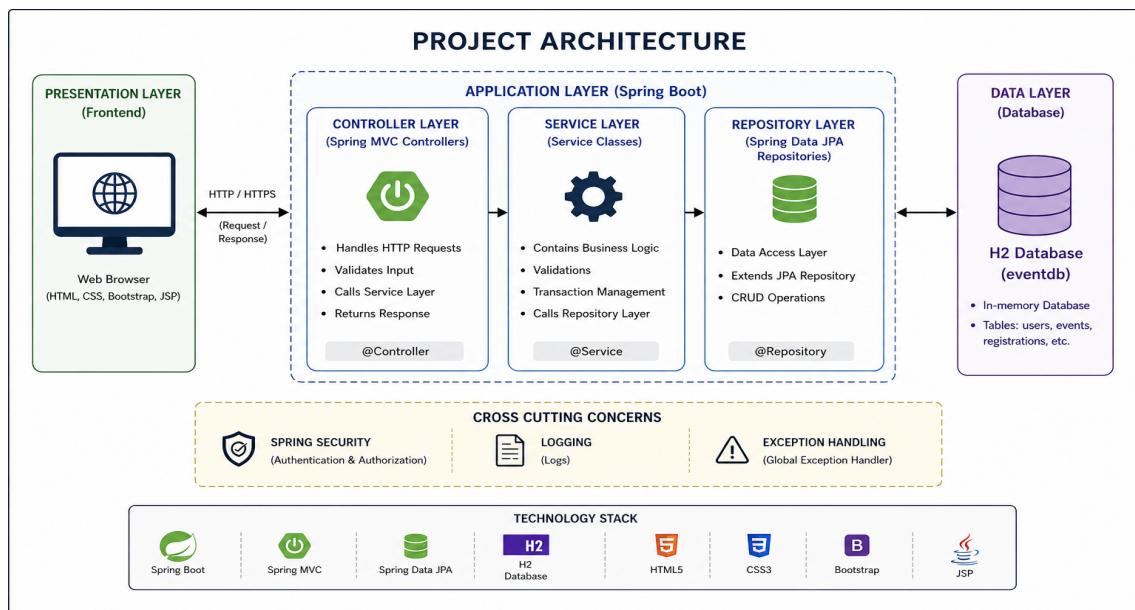


Figure 4.1: Project Architecture

[Figure 4.1] shows the project architecture divided into presentation, application, and data layers. The frontend interacts with Spring Boot controllers, which pass requests through service and repository

layers to handle business logic and database operations. Cross-cutting features like security, logging, and exception handling ensure reliable and secure system functionality.

4.2 Data Flow Diagram

The data flow of the Smart Campus Event Management System begins when the user (student or admin) registers or logs into the system through the login module using valid credentials. After successful authentication, the user performs actions such as viewing events, registering for events, or providing feedback, which are treated as inputs to the system. These inputs are processed by the Spring Boot backend, where controllers and services validate the data and execute the required business logic. The processed data is then stored in or retrieved from the MySQL database using JPA/Hibernate, ensuring efficient data management. Finally, the system generates a response, which is displayed to the user in the form of updated event details or confirmation messages, ensuring a smooth and continuous flow of data throughout the application. Cross-cutting concerns such as authentication and authorization using Spring Security ensure that only authorized users can access specific features, while logging mechanisms track system activities for monitoring and debugging. Exception handling mechanisms

capture errors and provide meaningful responses to the user without breaking the system flow. Finally, the processed output is returned to the frontend and displayed as updated event listings, registration confirmations, or report summaries, ensuring a seamless, secure, and efficient data flow throughout the application.

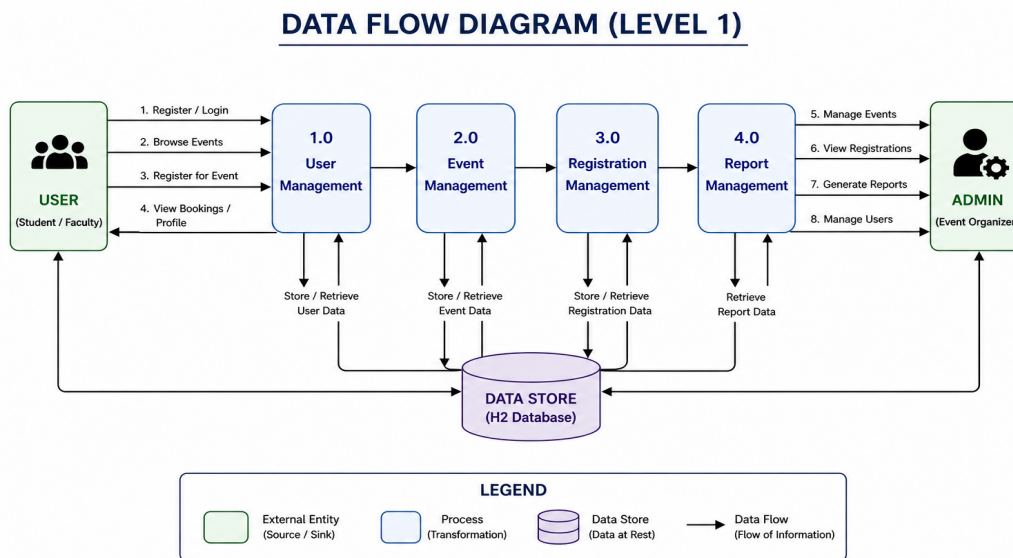


Figure 4.2: Data Flow Diagram

[Figure 4.2] shows the Level 1 data flow of the system, illustrating how users and administrators interact with modules such as user management, event management, registration, and report management. Each process exchanges data with the central H2 database to store and retrieve user, event, and registration information. The diagram highlights the flow of operations from login and event browsing to registration and report generation.

4.3 Module 1: User Module

This module handles all functionalities related to students. It allows users to log in, browse available events, filter events based on criteria such as date or department, register for events, and provide feedback. The module ensures proper validation of user inputs and maintains a record of user activities in the database.

4.4 Module 2:Admin Module

The admin module is responsible for managing the overall system. Administrators can securely log in and perform operations such as creating new events, updating event details, deleting events, and viewing registration statistics. This module ensures efficient event management and helps in monitoring user participation.

4.5 Module 3:Database Module

The database module manages data storage and retrieval. It includes entities such as User, Event, and Registration. All CRUD operations are performed using Spring Data JPA, ensuring efficient interaction with the MySQL database. This module maintains data integrity and supports features like event tracking and analytics.

Advantages of this Project

This project provides a streamlined and digital approach to managing campus event registrations

a) User-Friendly Interface:

The system offers an intuitive and responsive interface, making it easy for users to browse events and complete bookings quickly.

b)Real-Time Data Handling:

The application dynamically updates booking information, ensuring accurate and up-to-date ticket availability.

c)Centralized Data Management:

All booking and event details are stored in a structured database, simplifying data access and management.

Disadvantages The system has certain limitations that can be addressed in future improvements. It currently depends on a local server setup, limiting accessibility beyond a specific environment. Advanced features such as secure authentication, online payment integration, and scalability for large-scale events are not yet implemented. Additionally, performance optimization may be required for handling high traffic scenarios.

Chapter 5

RESULTS AND DISCUSSIONS

The Smart Campus Event Management Application was successfully designed and implemented to streamline the process of event registration and ticket booking within an academic environment. The system integrates a responsive frontend interface with a Node.js backend and MySQL database, ensuring efficient data handling and smooth user interaction.

The application allows users to browse events, select ticket types, and complete registrations seamlessly. The booking details are stored in the database and retrieved in real-time, demonstrating the reliability of the system. The confirmation module generates an e-ticket with booking details, improving user experience and reducing manual effort.

Testing results indicate that the system performs efficiently for small to medium-scale usage, with accurate data storage and retrieval. The frontend interface is responsive and user-friendly, while the backend

ensures proper validation and processing of booking data. Screenshots of the application interface, booking form, and database records are included below for reference.

Table 5.1: Application Feature Comparison

Feature	Eventbrite	BookMyShow	Proposed App
Custom Branding	Limited	Limited	High
Transaction Fees	High	Moderate	None (Self-Hosted)
Setup Complexity	High	Moderate	Low
Target Audience	Commercial Events	Entertainment Sector	Institutional / De- partmental
Data Ownership	Third-Party	Third-Party	Fully Owned
Architecture	Microservices	Microservices	Monolithic MVC
Cost Efficiency	Low	Moderate	High

The [Table 5.1] represents a comparative analysis of application features between Eventbrite, BookMyShow, and the proposed Smart Campus Event Management System. It highlights key differences in terms of customization, cost, architecture, and usability. Eventbrite and BookMyShow offer limited custom branding and involve transaction fees, whereas the proposed system provides high customization with no transaction costs due to its self-hosted nature. In terms of setup complexity, the proposed system is simpler and more suitable for institutional use compared to the relatively complex configurations of existing platforms.

5.1 Student Login

The screenshot displays a web interface for finding registered events. At the top, the heading "My Registered Events" is shown. Below it is a search box titled "Find Your Events" with the instruction "Enter your email to see the events you have registered for." The search box contains the email address "vtu25914@veltech.edu.in" and a "Find My Events" button. Below the search box, the results are displayed under the heading "Found 2 event(s)". There are two event cards. The first card is for a "Data Science & Analytics Masterclass" on Friday, 08 May, 2026, at 09:06 am, located in Computer Lab 1, Block A, under the Computer Science department. It shows a "Registered" status and a "Feedback" button. The second card is for a "Full Stack Web Development Workshop" on Friday, 15 May, 2026, at 09:06 am, located in IT Lab 2, Block B, under the Information Technology department. It also shows a "Registered" status and a "Feedback" button. The footer of the page reads "© 2025 Smart Campus Event Management System".

Event Title	Date & Time	Location	Department	Status	Action
Data Science & Analytics Masterclass	Fri, 08 May, 2026, 09:06 am	Computer Lab 1, Block A	Computer Science	Registered	Feedback
Full Stack Web Development Workshop	Fri, 15 May, 2026, 09:06 am	IT Lab 2, Block B	Information Technology	Registered	Feedback

Figure 5.1: Student Login Page

[Figure 5.1] shows the student login page where users can enter their email address to view the events they have registered for. The interface displays a list of registered events along with details such as date, location, and department. It also provides options like registration status and feedback for each event. The design is user-friendly and ensures easy navigation for students to access their event information.

5.2 Registration Page

The image shows a mobile application interface for event registration. At the top, there is a 'Back to Events' button. Below it, a green 'SEMINAR' tag is displayed. The main title is 'Entrepreneurship & Startup Innovation Seminar'. Event details include the date 'Sunday, 10 May 2026 - 09:06 AM', location 'Seminar Hall, Block C', category 'Management', and a registration count '0 / 150 registered'. A section titled 'About This Event' describes the seminar's focus on business model canvas, funding strategies, and startup methodology. A green 'Registration Open' button is present. Below this is a three-step progress indicator: 1. Details, 2. Verify OTP, and 3. Confirm. The registration form includes fields for 'Full Name *', 'Email Address *', and 'Department *'. The 'Full Name' field contains the placeholder 'Enter your full name'. The 'Email Address' field contains the placeholder 'your.email@college.edu'. The 'Department' field is a dropdown menu with the placeholder 'Select Department'. A large pink button at the bottom of the form is labeled 'Send OTP to Email'. The footer of the page contains the copyright notice '© 2025 Smart Campus Event Management System'.

Figure 5.2: Event Registration Page

[Figure 5.2] shows the event registration page where students can view detailed information about a selected event and complete the registration process. The form collects user details such as name, email, and department, and includes an OTP verification step for secure registration. This page ensures accurate data entry and provides a smooth and secure way for students to enroll in events.

5.3 Upcoming Events

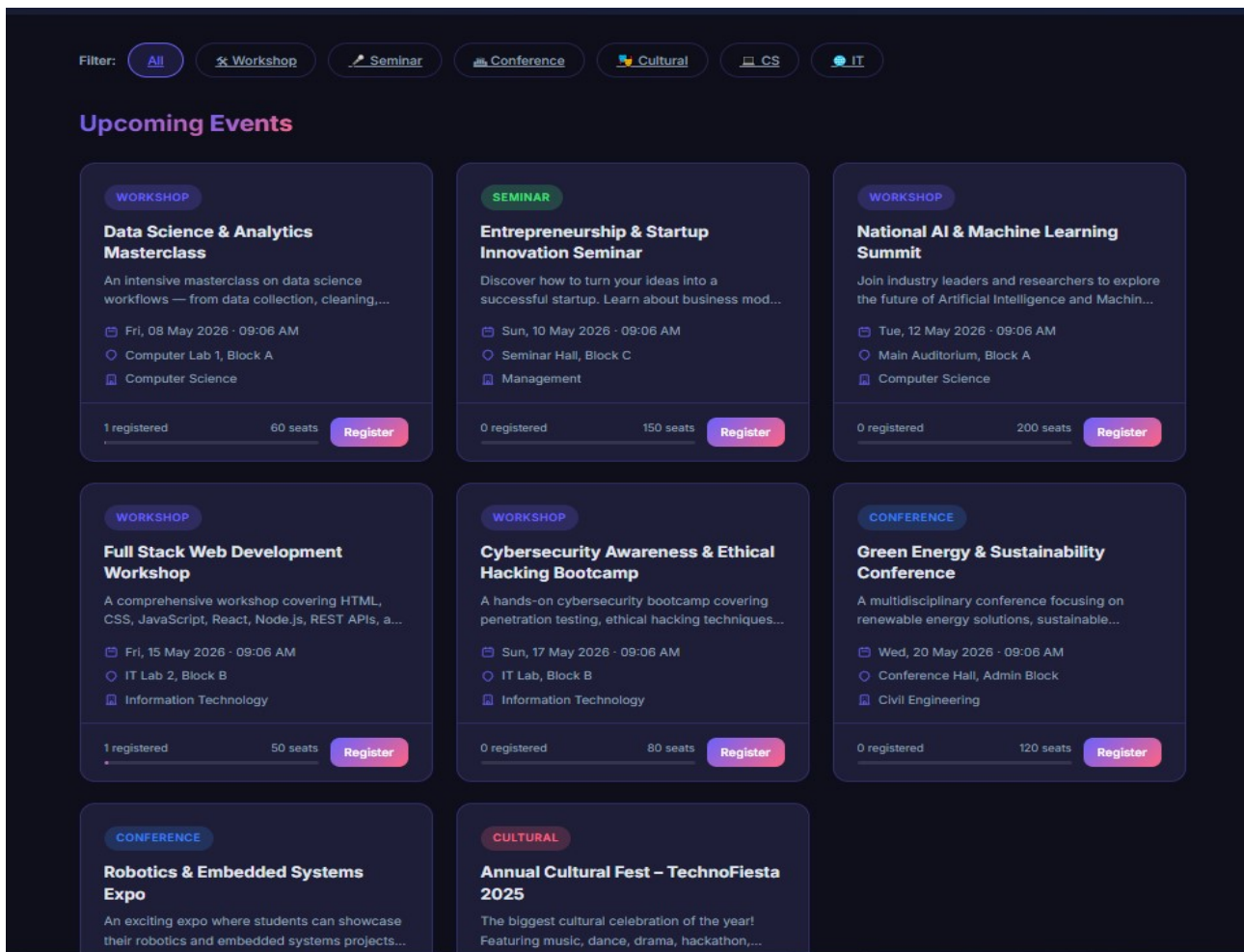


Figure 5.3: Upcoming Events

[Figure 5.3] shows the upcoming events page where students can browse a list of available events with details such as title, date, venue, and category. The interface provides filtering options to view events based on different types like workshops, seminars, and conferences. It also includes a registration button for each event, allowing students to quickly enroll in the events of their interest.

5.4 OTP Verification

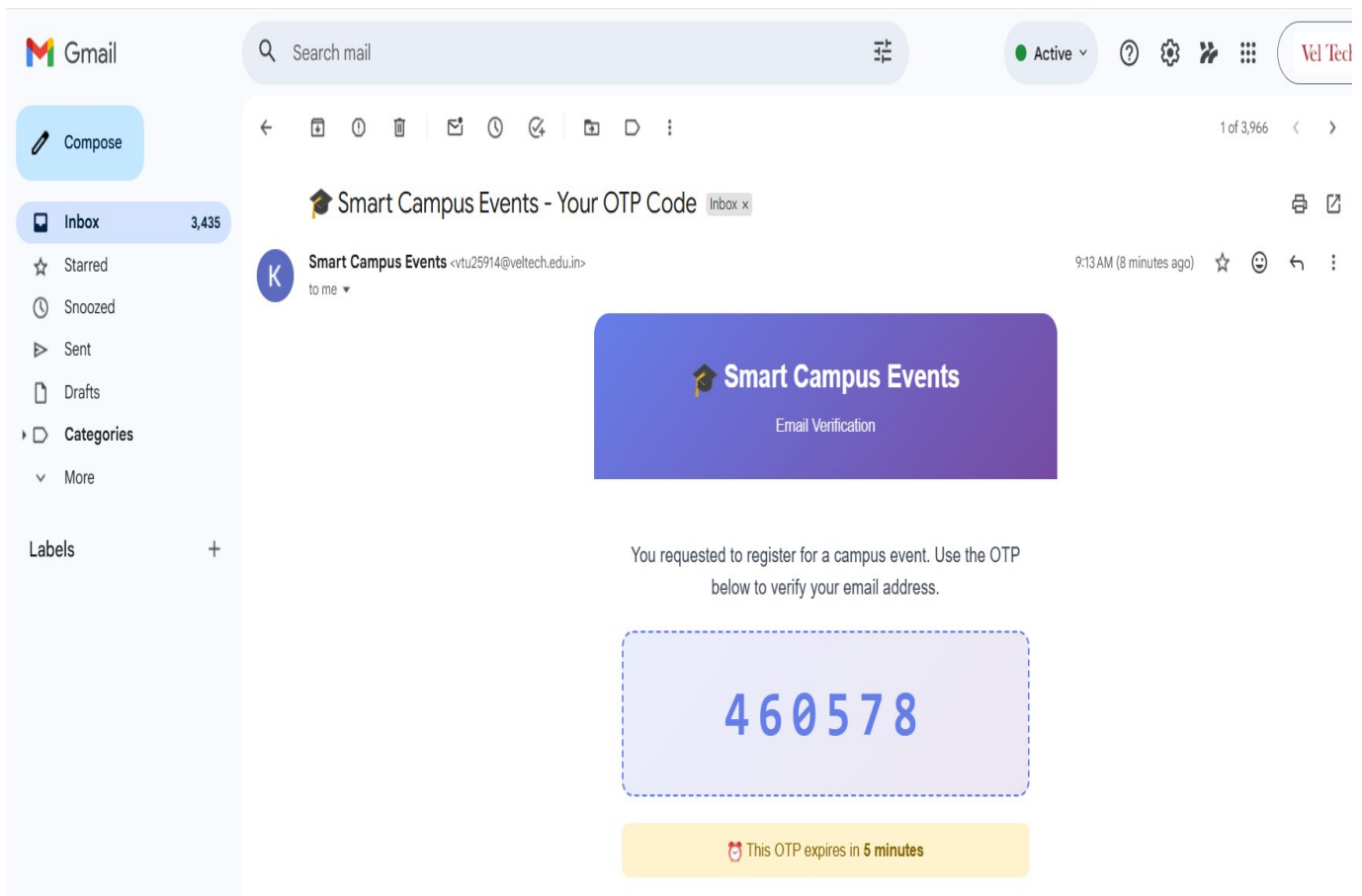
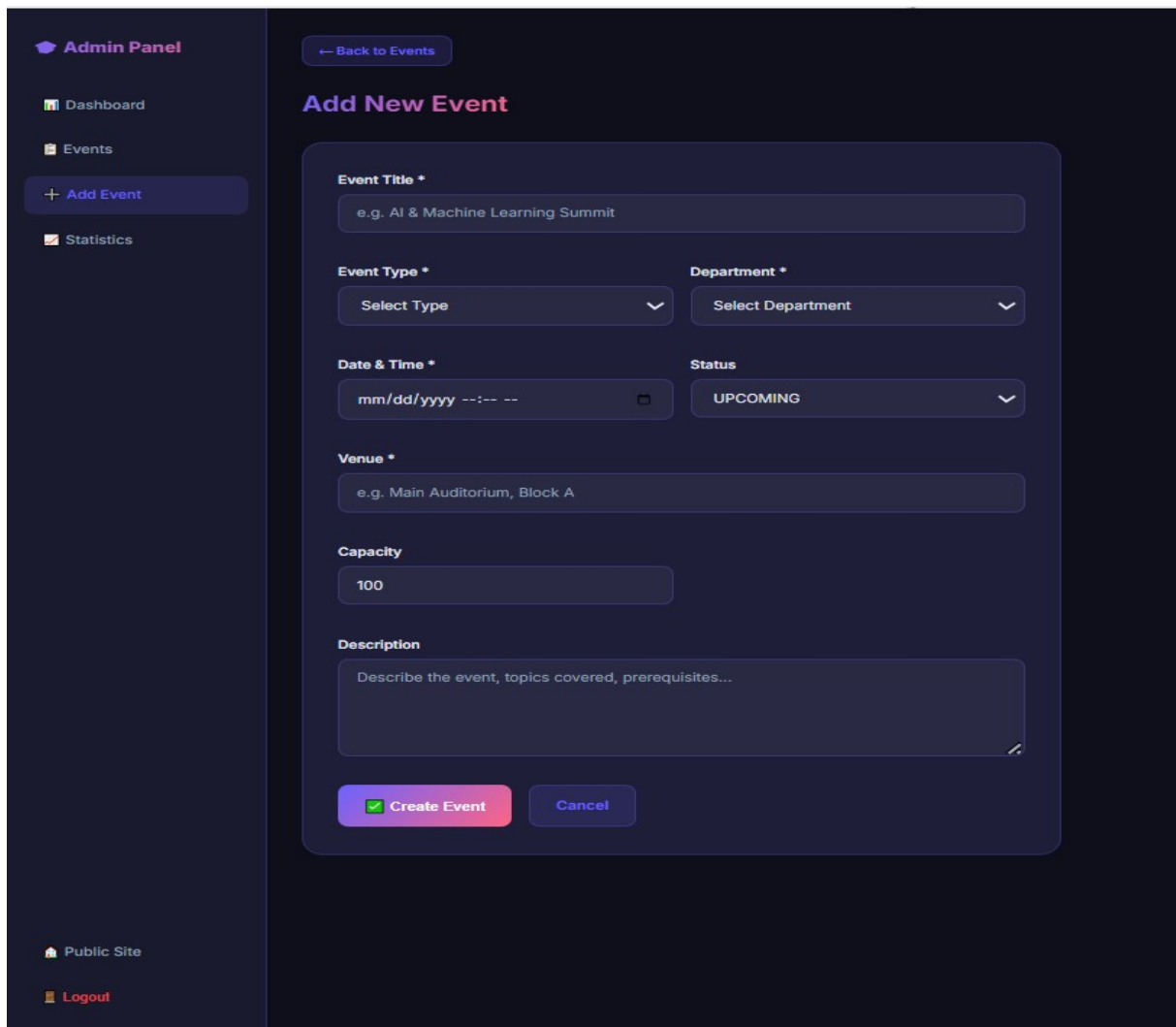


Figure 5.4: E-mail OTP Verification

[Figure 5.4] shows the Email OTP Verification page where users receive a one-time password (OTP) to verify their email during event registration. The interface displays the OTP sent to the user's email along with a message indicating its validity period. This step ensures secure authentication and confirms the user's identity before completing the registration process. It also prevents unauthorized access and enhances the overall security of the system.

5.5 Add new Event



The screenshot displays the 'Add New Event' form within an Admin Panel. The left sidebar contains navigation links: 'Admin Panel' (active), 'Dashboard', 'Events', '+ Add Event', and 'Statistics'. At the bottom of the sidebar are links for 'Public Site' and 'Logout'. The main content area features a 'Back to Events' button and the title 'Add New Event'. The form itself includes the following fields: 'Event Title *' (text input with placeholder 'e.g. AI & Machine Learning Summit'), 'Event Type *' (dropdown menu with 'Select Type'), 'Department *' (dropdown menu with 'Select Department'), 'Date & Time *' (datetime picker with placeholder 'mm/dd/yyyy --:-- --' and a calendar icon), 'Status' (dropdown menu with 'UPCOMING'), 'Venue *' (text input with placeholder 'e.g. Main Auditorium, Block A'), 'Capacity' (text input with '100'), and 'Description' (text area with placeholder 'Describe the event, topics covered, prerequisites...'). At the bottom of the form are two buttons: 'Create Event' (with a green checkmark icon) and 'Cancel'.

Figure 5.5: Add new Event

[Figure 5.5] shows the “Add New Event” page where the admin can enter details to create a new event. The interface includes fields such as event title, type, department, date and time, venue, capacity, and description. It also provides dropdown options for selecting event type, department, and status. The admin can submit the form using the “Create Event” button or cancel the operation.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Smart Campus Event Management Application was successfully designed and implemented to digitize and simplify event registration and ticket booking within an academic environment. The system integrates a responsive frontend built using HTML, CSS, and JavaScript with a Node.js and Express backend, along with a MySQL database for efficient data storage. The application enables users to browse events, select ticket types, and complete bookings through an intuitive interface. It ensures proper validation of user inputs, accurate calculation of ticket amounts, and reliable storage of booking details including event selection. The seamless communication between frontend and backend through API requests ensures real-time data handling and smooth user experience. Compared to existing large-scale plat-

forms, the proposed system is lightweight, cost-effective, and specifically tailored for campus-level events. It reduces manual effort, minimizes errors, and provides a structured approach to managing event registrations efficiently.

6.2 Future Enhancements

The current system lays a strong foundation but can be significantly enhanced to create a more powerful and scalable platform. Future improvements include:

- Implementing user authentication and login system for secure and personalized access
- Integrating online payment gateways for real-time ticket booking and transactions
- Adding email and SMS notifications for instant booking confirmation and reminders
- Development of an admin dashboard for managing events, tracking bookings, and analytics
- Implementing QR code-based e-ticket verification for faster check-in.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of Project to Complex Engineering Problems (CEP)

Table 7.1: Mapping of Project to Complex Engineering Problems (CEP)

Level	Attribute	Characteristics	Justification
WP1	Depth of Knowledge	Requires engineering knowledge across multiple domains	Full stack development involving frontend, backend, and database technologies
WP2	Conflicting Requirements	Technical and usability trade-offs	Balancing performance, user experience, and data consistency
WP3	Depth of Analysis	Analytical and design thinking	Designing APIs, database schema, and system workflows
WP4	Familiarity	Partially new problem domain	Customized solution for campus-specific event management
WP5	Applicable Codes	Limited standardization	Custom booking logic without strict external standards
WP6	Stakeholder Needs	Multiple stakeholders involved	Students, organizers, and administrators interacting with system
WP7	Interdependence	Integration across multiple layers	Strong dependency between frontend, backend, and database

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Use of diverse technologies	HTML, CSS, JavaScript, Node.js, Express, and MySQL used together
EA2	Level of Interactions	Complex module interactions	API-based communication between frontend, backend, and database
EA3	Innovation	Modern technologies	Real-time booking system with dynamic UI and database updates
EA4	Societal Impact	Impact on users	Improves efficiency of campus event management and reduces manual work
EA5	Familiarity	Beyond standard practices	Full stack system integrating multiple layers and real-time processing

7.3 SDG Mapping

Table 7.3: SDG Mapping for the Project

SDG No.	SDG Title	Relevance to the Project
SDG 4	Quality Education	Facilitates student participation in academic events, workshops, and seminars, enhancing learning beyond the classroom.
SDG 9	Industry, Innovation and Infrastructure	Promotes digital infrastructure and automation in campus systems
SDG 11	Sustainable Cities and Communities	Supports smart campus ecosystem through efficient event management
SDG 12	Responsible Consumption and Production	Reduces paper usage by enabling digital ticket booking
SDG 16	Peace, Justice and Strong Institutions	Ensures secure data handling and reliable system operations

REFERENCES

- [1] Pivotal Software Inc., “Spring Boot: Building Production-Ready Applications,” Spring Boot Official Documentation, 2024. [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/html/>
- [2] VMware Inc., “Spring Framework: Core Features and MVC Architecture,” Spring Framework Documentation, 2024. [Online]. Available: <https://docs.spring.io/spring-framework/docs/current/reference/html/web.html>
- [3] Oracle Corporation, “Java Platform Standard Edition Documentation,” Oracle Java Documentation, 2024. [Online]. Available: <https://docs.oracle.com/en/java/javase/>
- [4] Oracle Corporation, “MySQL Database Management System Documentation,” MySQL Official Documentation, 2024. [Online]. Available: <https://dev.mysql.com/doc/>

- [5] Thymeleaf Team, “Thymeleaf: Modern Server-Side Java Template Engine,” Thymeleaf Documentation, 2024. [Online]. Available: <https://www.thymeleaf.org/documentation.html>
- [6] Hibernate Organization, “Hibernate ORM Documentation,” Hibernate Official Documentation, 2024. [Online]. Available: <https://hibernate.org/orm/documentation/>
- [7] OpenJS Foundation, “HTML5 and CSS3 Web Standards,” MDN Web Docs, 2024. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web>
- [8] OWASP Foundation, “Web Application Security Guidelines,” OWASP Top Ten Project, 2024. [Online]. Available: <https://owasp.org/www-project-top-ten/>
- [9] R. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. dissertation, University of California, Irvine, 2000. [Online]. Available: <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>